Software Requirements Specification — Online Examination Portal

Course: UE24CS341A — Software Engineering, PES University, Dept. of CSE
Project: Online Examination System with Multiple-Choice Questions and Evaluation
Team: Kshitij G Shettigar (Backend & Data) · Kartik Basavaraj Meti (Frontend & UX) · Komal S Sajja (QA & Docs)
Version: 3.0 · **Date:** 2026-10-15

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements of the Online Examination Portal. It is the single source for the GitHub backlog (one backlog item per requirement), the Software Architecture Document (`Documents/SAD.md`) and the Test Plan (`Documents/Test_Plan.md`).

1.2 Scope

Faculty author multiple-choice questions into a tagged question bank, assemble them into an examination and schedule it for a fixed window. Enrolled Students attempt the examination under a server-enforced timer with autosave and resume. Every submission is evaluated automatically against the stored answer key, and results are released when Faculty publishes them.

Out of scope: descriptive answers and manual grading · webcam or AI proctoring · biometric identity verification · payments · adaptive testing · native mobile applications · offline mode · plagiarism detection.

1.3 Actors

Actor	Description
Student	An enrolled candidate who sits examinations and views published results
Faculty	Authors questions and examinations, schedules them, publishes results
Administrator	Manages courses, enrolls students and reviews the audit trail
Clock	The server clock — triggers automatic submission when an attempt's time runs out

1.4 How to read the requirement tables

The tables follow the course requirements-table template: **Req ID, Type, Description ("The system shall …"), Priority (High / Medium / Low), Acceptance Criteria (measurable pass/fail), Rationale and Comments**. The Comments column names the use case that realises the requirement and the sprint that implements it. Requirement IDs are sequential with no gaps: `FR-001` … `FR-030` and `NFR-001` … `NFR-014`.

2. Functional Requirements

2.1 User Management & Authentication

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale (short)	Comments
FR-001	Functional	The system shall allow a visitor to register a Student account by submitting full name, institutional email, roll number and password.	High	Pass: a submission with all four valid fields creates a Student account; a missing field, a non-institutional email, or a duplicate email or roll number is rejected with an inline error and no account is created.	Students must be able to join without waiting for an administrator.	UC-01 · Sprint 1
FR-002	Functional	The system shall authenticate a user against email and password and establish a session on success.	High	Pass: correct credentials open the dashboard for the user's role; a wrong password shows "Invalid email or password" and no session is created.	Every other function depends on a known, signed-in user.	UC-02 · Sprint 1
FR-003	Functional	The system shall assign exactly one role — Student, Faculty or Administrator — to every account, and shall restrict every screen and endpoint to the roles authorised for it.	High	Pass: a Student requesting a Faculty URL directly receives HTTP 403; an unauthenticated request to any protected URL is redirected to sign-in (HTTP 302) or refused (HTTP 401).	Hiding a button is not access control; the server must refuse.	UC-02 · Sprint 1

2.2 Course & Enrollment Management

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale (short)	Comments
FR-004	Functional	The system shall allow an Administrator to create, edit and archive a Course identified by course code, title and semester, and to assign one or more Faculty accounts to it.	High	Pass: a Course created with code, title and semester appears in the course list with its assigned Faculty; a duplicate course code is rejected; an archived Course accepts no new examinations.	Examinations, questions and enrollments all hang off a Course.	UC-03 · Sprint 1
FR-005	Functional	The system shall allow Faculty or an Administrator to enroll Student accounts into a Course individually.	High	Pass: after enrolling a Student, that Student appears on the Course roster; enrolling the same Student twice leaves exactly one enrollment.	Enrollment decides who may sit which examination.	UC-04 · Sprint 1
FR-006	Functional	The system shall permit a Student to view and attempt only those examinations that belong to Courses in which the Student is enrolled.	High	Pass: a Student not enrolled in the Course does not see its examination and receives HTTP 403 on a direct URL to it.	Stops students sitting papers of courses they are not registered for.	UC-04 · Sprint 1

2.3 Question Bank Management

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale (short)	Comments
FR-007	Functional	The system shall allow Faculty to create a single-correct multiple-choice question consisting of a stem, between 2 and 6 options, and exactly one option marked correct.	High	Pass: a question with 4 options and one correct option is saved; 1 option, 7 options, zero correct or two correct options are each rejected with an error.	MCQ authoring is the core content of the system.	UC-05 · Sprint 1
FR-008	Functional	The system shall require every question to carry a Course, a topic label, a difficulty of Easy, Medium or Hard, positive marks, and negative marks.	High	Pass: saving a question with any of the five fields missing, a difficulty outside the three values, or negative positive-marks is rejected.	Tags make the bank searchable; marks drive scoring.	UC-05 · Sprint 1
FR-009	Functional	The system shall prevent modification or deletion of a question that belongs to a published or attempted examination, and shall instead create a new version of that question.	Medium	Pass: editing a question used in a published examination creates version 2; the examination still shows version 1 unchanged; deleting it is refused.	Changing a question under a live paper would make past scores wrong.	UC-05 · Sprint 2

2.4 Examination Authoring

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale (short)	Comments
FR-010	Functional	The system shall allow Faculty to create an Examination with a title, Course, duration in minutes, declared total marks, instructions, and the policy values: negative marking (on/off and value), question and option shuffling, and pass mark.	High	Pass: an Examination saved with all fields is listed in Draft state with the entered values; a duration of 0 or a pass mark above 100% is rejected.	Each paper needs its own rules.	UC-06 · Sprint 1
FR-011	Functional	The system shall allow Faculty to add questions to an Examination by selecting them from the question bank.	High	Pass: questions selected from the bank appear in the Examination in the selected order; a question from another Course cannot be added.	Papers are built by reuse, not retyping.	UC-06 · Sprint 1
FR-012	Functional	The system shall hold an Examination in Draft state until Faculty publishes it, and shall make only published Examinations visible to Students.	High	Pass: a Draft examination is not visible on any Student dashboard; after Publish it is visible to enrolled Students.	Faculty must be able to prepare a paper privately.	UC-06 · Sprint 1

2.5 Scheduling & Access Control

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale (short)	Comments
FR-013	Functional	The system shall allow Faculty to schedule a published Examination by setting a window start date-time and end date-time.	High	Pass: the window is saved and shown on the Examination; an end time earlier than the start time is rejected.	The window decides when the paper can be taken.	UC-07 · Sprint 1
FR-014	Functional	The system shall present to each Student a dashboard listing the Upcoming, Live and Completed Examinations of the Courses in which	High	Pass: with server time before, inside and after the window, the examination appears under Upcoming, Live and Completed respectively.	Students need one place to find their papers.	UC-07 · Sprint 2

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale (short)	Comments
		the Student is enrolled.				
FR-015	Functional	The system shall permit a Student to begin an attempt only when the current server time lies within the window, the Student is enrolled, and the Student has no previously submitted attempt for that Examination.	High	Pass: Start is refused with a message before the window opens, after it closes, for a non-enrolled Student and for a second attempt; it succeeds otherwise.	Enforces fairness: same window, one attempt each.	UC-08 · Sprint 1

2.6 Examination Delivery (Attempt)

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale (short)	Comments
FR-016	Functional	The system shall create an attempt record stamped with the server start time when a Student begins an Examination, and shall deliver the paper only after that record exists.	High	Pass: the attempt row with a server start time exists before the first question is returned; no question is returned if the row cannot be created.	No one can see the paper without a timed attempt on record.	UC-08 · Sprint 1
FR-017	Functional	The system shall display a countdown timer whose remaining time is derived from the server clock and the attempt start time, and shall not rely on the client clock.	High	Pass: moving the client PC clock forward by 30 minutes does not change the remaining time shown after a page reload.	A client clock can be tampered with.	UC-08 · Sprint 1
FR-018	Functional	The system shall display one question at a time with a navigation palette showing each question as Not Visited, Not Answered, Answered or Marked for Review.	High	Pass: answering, skipping and marking questions changes each palette cell to the matching state, and the states survive a page reload.	Lets students pace and review a long paper.	UC-08 · Sprint 2
FR-019	Functional	The system shall persist every response to the server within 5 seconds of the Student selecting it, without any explicit save action.	High	Pass: 5 seconds after selecting an option, the response is present in the database; killing the browser then loses no response older than 5 s.	A dropped connection must not cost a student their answers.	UC-09 · Sprint 2
FR-020	Functional	The system shall automatically submit an attempt, retaining all responses persisted up to that instant, when the remaining time reaches zero.	High	Pass: an attempt left open past its duration is in Submitted state with all saved responses, and no response saved after expiry is accepted.	Time limits must be enforced the same for every student.	UC-10 · Sprint 1
FR-021	Functional	The system shall allow a Student whose session was interrupted to resume the same attempt, with remaining time computed from the original server start time and all saved responses restored.	High	Pass: closing the browser 10 minutes into a 60-minute attempt and reopening it 5 minutes later shows about 45 minutes left and every earlier answer.	Interruptions happen; they must neither erase work nor add time.	UC-09 · Sprint 2

2.7 Automatic Evaluation & Scoring

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale (short)	Comments
FR-022	Functional	The system shall evaluate every submitted attempt automatically, without any human intervention.	High	Pass: every attempt in Submitted state, whether submitted manually or automatically, has a stored score without	Removes manual grading entirely.	UC-11 · Sprint 1

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale (short)	Comments
				any Faculty action.		
FR-023	Functional	The system shall award the question's full positive marks for a correct response, deduct the configured negative marks for an incorrect response, and award zero with no deduction for an unanswered question.	High	Pass: with +2 / −0.5 marking, 10 correct, 4 wrong and 6 unanswered scores exactly 18.0.	The scoring rule must be exact and predictable.	UC-11 · Sprint 1
FR-024	Functional	The system shall compute, for every evaluated attempt, the total score, the percentage and a Pass or Fail outcome against the Examination's pass mark.	High	Pass: 20 out of 50 with a 40% pass mark is stored as 40.0% and Pass; 19.5 out of 50 is 39.0% and Fail.	Students and Faculty need the outcome, not only the raw score.	UC-11 · Sprint 2

2.8 Results

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale (short)	Comments
FR-025	Functional	The system shall withhold every evaluated result from the Student until Faculty publishes the results of that Examination.	High	Pass: before publication the Student sees only "Submitted" with no score, and the results URL returns HTTP 403.	Faculty decide when results are released.	UC-12 · Sprint 2
FR-026	Functional	The system shall allow Faculty to publish and to unpublish the results of an Examination.	High	Pass: after Publish the scorecard is visible to the Student; after Unpublish it is hidden again.	Faculty need to correct a premature release.	UC-12 · Sprint 2
FR-027	Functional	The system shall display to a Student, for a published result, the total score, the percentage, the Pass or Fail outcome and the marks obtained on each question.	High	Pass: every figure on the scorecard equals the stored evaluation exactly, including per-question marks.	Students should see how the score was reached.	UC-12 · Sprint 2

2.9 Integrity & Audit

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale (short)	Comments
FR-028	Functional	The system shall record, with a server-side timestamp, every occurrence of the examination window losing focus during an active attempt.	Medium	Pass: switching tabs three times during an attempt creates three focus-loss records with server timestamps, visible to Faculty.	Gives Faculty evidence; a human decides.	UC-13 · Sprint 2
FR-029	Functional	The system shall exclude all correct-answer data from every response delivered to a Student during an active attempt.	High	Pass: the page source and every network response of an attempt contain no correct-option field or value.	Leaking the key in the page would void the exam.	UC-13 · Sprint 1
FR-030	Functional	The system shall write an append-only audit entry recording the actor, action, affected entity, timestamp and changed values for every creation, modification and deletion of a user, question, examination, answer key or result.	Medium	Pass: editing a question creates one audit row with old and new values; no screen or endpoint can edit or delete an audit row.	Disputes need an untampered record of who changed what.	UC-13 · Sprint 2

Functional requirement count: 30 — Sprint 1: 19 · Sprint 2: 11.

3. Non-Functional Requirements

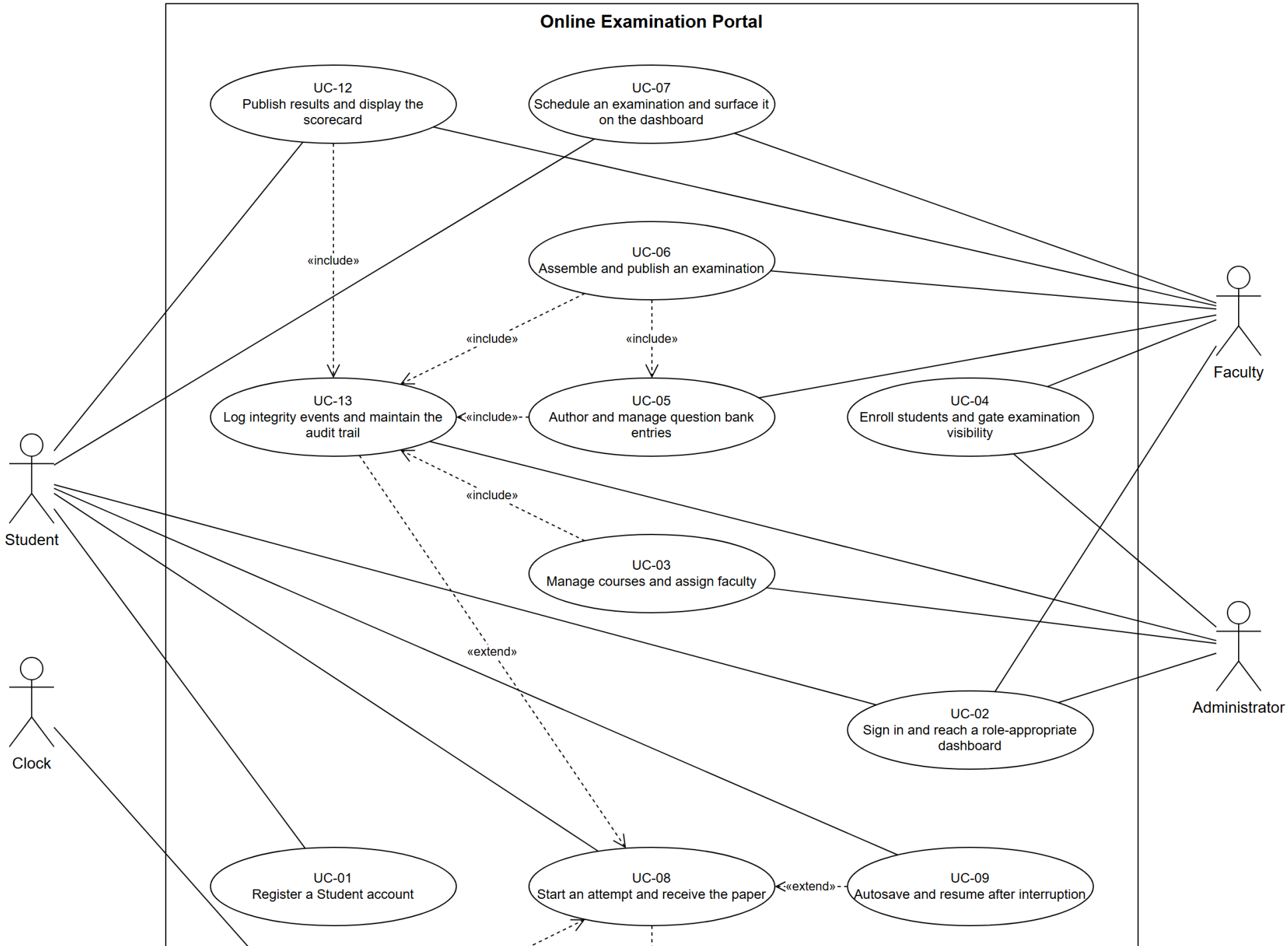
Req ID	Type	Description	Priority	Acceptance Criteria	Rationale (short)	Comments
NFR-001	Non-functional (Performance)	The system shall complete 95% of page loads and requests within 2 seconds while 200 attempts are active concurrently.	High	Pass: a Locust load test with 200 simulated students shows a 95th-percentile response time ≤ 2 s.	A slow paper costs students time.	Sprint 2
NFR-002	Non-functional (Performance)	The system shall evaluate a submitted 100-question attempt within 5 seconds.	High	Pass: a timed test of evaluating a 100-question attempt completes in ≤ 5 s.	Results must be ready right after the exam.	Sprint 2
NFR-003	Non-functional (Capacity)	The system shall sustain at least 200 concurrent attempts and a question bank of at least 50,000 questions without breaching NFR-001.	Medium	Pass: NFR-001's load test passes with the bank seeded to 50,000 questions.	A department-wide exam must not degrade the system.	Sprint 2
NFR-004	Non-functional (Reliability)	After an unplanned server restart, the system shall restore service within 60 seconds, with every in-progress attempt resumable with its saved responses and correct remaining time.	High	Pass: killing and restarting the server during an attempt restores service in ≤ 60 s and the attempt resumes intact.	A crash must not void an examination.	Sprint 2
NFR-005	Non-functional (Portability)	The system shall work on the two latest versions of Chrome, Edge and Firefox, and remain usable at viewport widths of 360 px and above.	Medium	Pass: the attempt flow completes on each browser and at 360 px width without horizontal scrolling.	Students use their own devices.	Sprint 2
NFR-006	Non-functional (Organisational)	All code and documents shall be held in one Git repository; work shall proceed on feature branches; every merge to `main` shall be through a pull request raised by the assigned member and reviewed by another member.	High	Pass: the GitHub history shows no direct commits to `main` after 19 Oct, and every merged PR has a reviewer other than its author.	Required by the course; keeps `main` working.	Sprint 1
NFR-007	Non-functional (Organisational)	Every functional requirement shall be linked to at least one test case in the Test Plan.	Medium	Pass: the coverage table of `Documents/Test_Plan.md` lists all 30 FRs.	Untested requirements are unverified claims.	Sprint 2
NFR-008	Non-functional (Legal)	Personal data shall be handled per the Digital Personal Data Protection Act, 2023: consent at registration, collection only for examination use, and erasure on request within 30 days.	Low	Pass: registration cannot complete without ticking consent; an Administrator can erase a Student's personal data.	Indian law applies to student data.	Sprint 2
NFR-009	Non-functional (Accessibility)	The attempt flow shall meet WCAG 2.1 Level AA: fully keyboard-operable, text contrast at least 4.5:1, and every option control labelled for screen readers.	Medium	Pass: an axe/Lighthouse audit reports no AA violations, and the whole attempt can be completed with the keyboard alone.	Every student must be able to sit the exam.	Sprint 2
NFR-010	Non-functional (Maintainability)	Examination policy values — window, pass mark and negative marking — shall be configurable per examination and shall not be hard-coded.	Medium	Pass: code review finds no hard-coded pass mark or negative-marking value; changing them on an examination changes its scoring.	Departments use different rules.	Sprint 2

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale (short)	Comments
NFR-011	Non-functional (Security)	Passwords shall be stored only as salted hashes using an adaptive algorithm, and shall never be stored or logged in plain text.	High	Pass: the password column holds only `pbkdf2_sha256$…` hashes, and the logs contain no password after a registration and a sign-in.	A database leak must not leak passwords.	Sprint 1
NFR-012	Non-functional (Security)	The system shall lock an account for 15 minutes after 5 failed sign-in attempts within 15 minutes.	Medium	Pass: the 6th sign-in attempt after 5 failures is refused even with the right password, and succeeds after 15 minutes.	Blocks password guessing.	Sprint 2
NFR-013	Non-functional (Security)	Every state-changing request shall carry an anti-CSRF token, all database access shall go through the ORM (no string-built SQL), and error pages shall show no stack trace or internal detail.	High	Pass: a POST without a CSRF token is refused with HTTP 403; a code search finds no raw SQL; a forced error shows a generic page with `DEBUG=False`.	Covers the most common web attacks.	Sprint 1
NFR-014	Non-functional (Security)	Secrets such as the Django SECRET_KEY and database credentials shall not be committed to the repository; they shall be read from environment variables.	High	Pass: a secret scan of the public repository finds no secret, and the app refuses to start without the SECRET_KEY variable.	The repository is public.	Sprint 1

Non-functional requirement count: 14 — Sprint 1: 4 · Sprint 2: 10.

4. Use Case Model

4.1 Use case diagram


Online Examination Portal
UC-12
Publish results and display the scorecard
UC-07
Schedule an examination and surface it on the dashboard
«include»
UC-06
Assemble and publish an examination
«include»
«include»
UC-13
Log integrity events and maintain the audit trail
«include»
UC-05
Author and manage question bank entries
UC-04
Enroll students and gate examination visibility
«include»
UC-03
Manage courses and assign faculty
«extend»
UC-02
Sign in and reach a role-appropriate dashboard
UC-01
Register a Student account
UC-08
Start an attempt and receive the paper
«extend»
UC-09
Autosave and resume after interruption
Student
Clock
Faculty
Administrator

Source: `Documents/diagrams/use_case_diagram.drawio` (open in draw.io).

ID	Use case	Primary actor	Requirements
UC-01	Register a Student account	Student	FR-001
UC-02	Sign in and reach a role-appropriate dashboard	Student, Faculty, Administrator	FR-002, FR-003
UC-03	Manage courses and assign faculty	Administrator	FR-004
UC-04	Enroll students and gate examination visibility	Faculty, Administrator	FR-005, FR-006
UC-05	Author and manage question bank entries	Faculty	FR-007, FR-008, FR-009
UC-06	Assemble and publish an examination	Faculty	FR-010, FR-011, FR-012
UC-07	Schedule an examination and surface it on the dashboard	Faculty (Student views)	FR-013, FR-014
UC-08	Start an attempt and receive the paper	Student	FR-015, FR-016, FR-017, FR-018
UC-09	Autosave and resume after interruption	Student	FR-019, FR-021
UC-10	Automatic submission on timer expiry	Clock	FR-020
UC-11	Evaluate a submitted attempt automatically	— (system behaviour)	FR-022, FR-023, FR-024
UC-12	Publish results and display the scorecard	Faculty, Student	FR-025, FR-026, FR-027
UC-13	Log integrity events and maintain the audit trail	Administrator	FR-028, FR-029, FR-030

4.2 Relationships

From	Type	To	Justification
UC-06	«include»	UC-05	FR-011 — a paper is assembled by selecting bank questions
UC-08	«include»	UC-11	FR-022 — every submitted attempt is evaluated automatically
UC-10	«include»	UC-11	An auto-submitted attempt is evaluated on the same path as a manual one
UC-09	«extend»	UC-08	Extension point *session interrupted* (FR-021)
UC-10	«extend»	UC-08	Extension point *remaining time reaches zero* (FR-020)
UC-13	«extend»	UC-08	Extension point *window loses focus* (FR-028)
UC-03, UC-05, UC-06, UC-12	«include»	UC-13	FR-030 — every change to stored data writes an audit entry

Signing in (UC-02) is a pre-condition of every use case except UC-01 (FR-003). It is stated once here rather than drawn as eleven identical «include» arrows.

5. Use Case Flows

5.1 How to read a flow

Section	What it holds
Main success scenario	The path when nothing goes wrong. Numbered, alternating actor step and system step.
Alternate flows (A)	A different but still *successful* path, or a recoverable rejection. Numbered against the main step it branches from — `A1 (step 5)` branches at step 5.

Section	What it holds
Exception flows (E)	The use case cannot complete. The system leaves no partial state behind.
Business rules (BR)	Constraints that hold across every path, not steps in any one of them.

Two conventions hold throughout and are not repeated in each use case:

- **Authorisation.** Except for UC-01, every use case has the implicit pre-condition that the actor is signed in through UC-02 and holds the role named as its primary actor. FR-003 restricts every screen *and* every API endpoint, so a direct API call by the wrong role is refused with HTTP 403 and a direct call by an unauthenticated client with HTTP 401 — verified by IT_004, IT_005 and IT_006.
- **Audit.** Every use case that creates, modifies or deletes a user, question, examination, answer key or result writes an append-only audit entry as part of the same transaction. FR-030, and UC-13.

UC-01 — Register a Student account

Use case ID	UC-01
Primary actor	Student (as an unauthenticated visitor)
Secondary actors	—
Requirements	FR-001
Stakeholders	*Student* — wants an account without waiting on an administrator. *Institution* — wants only its own members admitted, and personal data handled lawfully (NFR-008).
Trigger	The visitor selects **Register** on the portal landing page.
Pre-conditions	The portal is reachable. No account exists for the visitor's institutional email address or roll number.
Post-conditions (success)	A Student account exists and can sign in immediately. The password is stored as a salted hash. Consent is recorded. An audit entry is written.
Post-conditions (failure)	No account, and no partial row, exists. Nothing about an existing account is disclosed or changed.
Frequency	High at the start of a semester, then rare.

Main success scenario

1. The visitor opens the registration page.
2. The system presents a form asking for full name, institutional email address, roll number and password, together with the data-processing consent statement required by NFR-008.
3. The visitor enters the four values and gives consent.
4. The visitor submits the form.
5. The system validates that all four fields are present, that the email is well formed and on the configured institutional domain, that neither the email nor the roll number is already registered, and that the password meets the configured policy.
6. The system creates the account with role **Student** and stores the password as a salted hash.

7. The system records the consent and writes an audit entry.
8. The system confirms *"Registration successful"* and offers the sign-in page.

Alternate flows

- **A1 (step 5) — a mandatory field is empty.** The system rejects the submission, marks the offending field with an inline error, creates no account, and returns the visitor to step 3 with the other entries preserved. UT_002
- **A2 (step 5) — the email is not on the institutional domain.** Rejected with *"Please register with your institutional email address."* Return to step 3. UT_003
- **A3 (step 5) — the email is malformed.** Rejected with *"Enter a valid email address."* Return to step 3. UT_004
- **A4 (step 5) — the email is already registered.** Rejected with *"An account with this email already exists."* The existing account is not modified in any way. UT_005
- **A5 (step 5) — the roll number is already registered.** Rejected with *"This roll number is already registered."* Return to step 3. UT_006

Exception flows

- **E1 (step 6) — two registrations for the same roll number arrive simultaneously.** Exactly one account is created; the other is rejected as a duplicate. No orphaned or partial row survives. IT_002
- **E2 (step 6) — the datastore is unavailable.** The transaction is rolled back, no account is created, and the visitor is told to retry. No confirmation is shown.

Business rules

- **BR-01.** The accepted institutional domain is configuration, not code (NFR-010).
- **BR-02.** The plaintext password appears nowhere — not in the database, not in application logs. IT_001
- **BR-03.** Self-registration yields the **Student** role and only that role. Faculty and Administrator accounts are provisioned, never self-served (FR-003).
- **BR-04.** A Student's personal data is erasable on request within 30 days (NFR-008).

Test cases: UT_001–UT_006, IT_001–IT_002, ST_001

UC-02 — Sign in and reach a role-appropriate dashboard

Use case ID	UC-02
Primary actor	Student, Faculty or Administrator
Requirements	FR-002, FR-003
Stakeholders	*User* — wants to reach their own work in one step. *Institution* — wants no account enumerable and no role able to reach another's data.
Trigger	The user opens the sign-in page, or requests a protected page while holding no session.
Pre-conditions	The account exists and is active.
Post-conditions (success)	A session is established, bound to exactly one role, with a session identifier regenerated at sign-in. The user is on the dashboard for that role.

Post-conditions (failure)	No session is established. The failure message does not reveal whether the email exists.
Frequency	Very high — every interaction with the portal begins here.

Main success scenario

9. The user opens the sign-in page.
10. The system presents fields for email address and password.
11. The user enters both and submits.
12. The system verifies the email against the stored account and the password against the stored salted hash.
13. The system regenerates the session identifier and establishes a session bound to the account's single role.
14. The system routes the user to the dashboard for that role — Student, Faculty or Administrator.
15. Every subsequent screen request and API call is authorised against that role for the life of the session.

Alternate flows

- **A1 (step 4) — the email is correct but the password is wrong.** Sign-in is refused with *"Invalid email or password."* No session is established. UT_008
- **A2 (step 4) — the email is not registered.** Refused with the **identical** message and a comparable response time, so accounts cannot be enumerated by probing. UT_009
- **A3 (step 3) — the password field is empty.** Refused client-side with *"Password is required."* No authentication attempt is recorded. UT_010

Exception flows

- **E1 (step 7) — a Student calls a Faculty-only endpoint directly, bypassing the UI.** HTTP 403; nothing is created and no Faculty data is returned in the body. IT_004
- **E2 (step 7) — a Faculty member calls an Administrator-only endpoint.** HTTP 403. IT_005
- **E3 (step 7) — an unauthenticated client calls any protected endpoint.** HTTP 401. No endpoint defaults to open. IT_006

Business rules

- **BR-01.** Exactly one role per account — never none, never two (FR-003). UT_011
- **BR-02.** The session identifier held before authentication is invalidated at sign-in, so a fixated session cannot be reused. IT_003
- **BR-03.** Authorisation is enforced at the API, not merely by hiding UI controls. Hiding a button is a usability choice; refusing the endpoint is the control. ST_002

Test cases: UT_007–UT_011, IT_003–IT_006, ST_002

UC-03 — Manage courses and assign faculty

Use case ID	UC-03
Primary actor	Administrator
Requirements	FR-004
Includes	UC-13 (audit entry)
Stakeholders	*Administrator* — wants the semester's course list to reflect the department's offering. *Faculty* — wants their assigned courses to appear on their dashboard.
Trigger	The Administrator opens **Course Administration**.
Pre-conditions	The Administrator is signed in. The Faculty accounts to be assigned already exist.
Post-conditions (success)	The course exists with its code, title and semester, and carries one or more assigned Faculty. Each assigned Faculty sees it. An audit entry is written.
Post-conditions (failure)	The course list is unchanged.
Frequency	Once per course per semester, plus occasional edits.

Main success scenario

16. The Administrator opens Course Administration.
17. The system lists the existing courses with their code, title, semester and assigned Faculty.
18. The Administrator selects **Create Course**.
19. The Administrator enters the course code, the title and the semester.
20. The Administrator assigns one or more Faculty accounts to the course.
21. The system validates the entry and persists the course together with its Faculty assignments.
22. The system writes an audit entry recording the actor, the action, the entity and the values.
23. The course appears in the list as active, and on the dashboard of every assigned Faculty member.

Alternate flows

- **A1 (step 3) — edit an existing course.** The Administrator opens a course, changes its title, semester or Faculty assignment, and saves. The system revalidates and writes an audit entry carrying both the previous and the new values.
- **A2 (step 3) — archive a course.** The course stops accepting new examinations and disappears from the active list. Existing examinations, attempts and results are preserved unchanged — an archive is not a delete, because results must survive for the audit trail.
- **A3 (step 5) — assign a further Faculty member later.** A course may carry more than one Faculty account; assignment is additive and reversible.

Exception flows

- **E1 (step 6) — the course code duplicates an existing course.** Rejected; the existing course is untouched.
- **E2 (step 6) — a mandatory field is missing.** Rejected with an inline error; nothing is persisted.

- **E3 (any step) — a Faculty member or Student reaches the course-administration endpoint directly.** HTTP 403 (FR-003). IT_005

Business rules

- **BR-01.** A course is identified by its course code, which is unique.
- **BR-02.** A course with no assigned Faculty is permitted at creation but cannot have an examination authored against it, because authoring requires an assigned Faculty member.
- **BR-03.** Course creation, edit and archive are Administrator-only. Faculty maintain content within a course, not the course itself.

Test cases: UT_012–UT_016, IT_007–IT_008, ST_003

UC-04 — Enroll students and gate examination visibility

Use case ID	UC-04
Primary actor	Faculty or Administrator
Requirements	FR-005, FR-006
Stakeholders	*Faculty* — wants their cohort in place before an examination opens. *Student* — wants to see their own courses and nobody else's.
Trigger	Faculty or the Administrator opens the enrollment view of a course.
Pre-conditions	The course exists and is active. The Student accounts exist. The actor is an Administrator, or Faculty assigned to that course.
Post-conditions (success)	The Student is enrolled in the course and can see and attempt its published, scheduled examinations.
Post-conditions (failure)	The enrollment list is unchanged, and visibility is unchanged.
Frequency	Concentrated at the start of a semester.

Main success scenario

24. The actor opens a course and selects **Enrollment**.
25. The system lists the Students currently enrolled.
26. The actor selects **Enroll Student** and identifies the Student by roll number or email address.
27. The system verifies that the account exists, holds the Student role, and is not already enrolled.
28. The system records the enrollment.
29. The Student's dashboard now lists that course, and the course's published, scheduled examinations become visible to that Student and to no one outside the enrollment.

Alternate flows

- **A1 (step 3) — enroll several Students in one pass.** The actor identifies each in turn; the system validates each independently, so one invalid entry does not discard the valid ones.

- **A2 (step 3) — remove an enrollment.** The Student loses visibility of that course's future examinations. Attempts and results already recorded are preserved, because removing a person from a list must not erase evidence of what they sat.

Exception flows

- **E1 (step 4) — the account does not exist.** Rejected; the actor is told no such Student was found.
- **E2 (step 4) — the account is Faculty or Administrator.** Rejected; only a Student role can be enrolled as a candidate.
- **E3 (step 4) — the Student is already enrolled.** Rejected as a duplicate; exactly one enrollment row is held per Student per course.
- **E4 (step 6) — a Student calls the examination endpoint for a course they are not enrolled in.** HTTP 403, and no examination metadata is returned. FR-006 is an access control, not a UI filter. IT_011

Business rules

- **BR-01.** A Student may view and attempt only examinations belonging to a course in which the Student is enrolled (FR-006).
- **BR-02.** Faculty may enroll Students only into a course they are assigned to; an Administrator may enroll into any course.
- **BR-03.** Enrollment gates *visibility*; it does not by itself permit an attempt. UC-08 applies the window, the enrollment and the no-prior-submission checks together (FR-015).

Test cases: UT_017–UT_021, IT_009–IT_011, ST_004

UC-05 — Author and manage question bank entries

Use case ID	UC-05
Primary actor	Faculty
Requirements	FR-007, FR-008, FR-009
Includes	UC-13 (audit entry)
Stakeholders	*Faculty* — wants a tagged, reusable pool rather than rewriting questions per paper. *Student* — depends on the answer key being right and on a live paper never changing beneath them.
Trigger	Faculty opens the question bank for a course and selects **New Question**.
Pre-conditions	Faculty is signed in and assigned to the course.
Post-conditions (success)	The question exists with a stem, 2 to 6 options, exactly one correct option, and its course, topic, difficulty, positive marks and negative marks. An audit entry is written.
Post-conditions (failure)	The bank is unchanged.
Frequency	High during preparation, then steady reuse.

Main success scenario

30. Faculty opens the question bank for a course.
31. The system lists the existing questions with their topic, difficulty and marks.
32. Faculty selects **New Question**.
33. Faculty enters the question stem.
34. Faculty enters between two and six options and marks exactly one of them correct.
35. Faculty sets the course, the topic label, the difficulty — Easy, Medium or Hard — the positive marks and the negative marks.
36. The system validates the option count, the single correct option and the presence of every mandatory attribute.
37. The system persists the question as version 1 and writes an audit entry.
38. The question becomes available for selection during examination assembly (UC-06).

Alternate flows

- **A1 (step 3) — edit a question not yet used.** The question belongs to no published or attempted examination, so it is edited in place. The audit entry records the previous and the new values. UT_068
- **A2 (step 3) — edit a question that* is* in a published or attempted examination. **The system refuses to modify it in place and instead creates a** new version**. The examination that already used the earlier version continues to reference that version, so a paper cannot change beneath a candidate mid-window. FR-009
- **A3 (step 3) — delete a question.** Permitted only while the question belongs to no published or attempted examination. Otherwise it is retired from selection, never removed.

Exception flows

- **E1 (step 7) — fewer than two or more than six options.** Rejected with the permitted range stated.
- **E2 (step 7) — no option, or more than one option, marked correct.** Rejected; a single-correct MCQ must carry exactly one key (FR-007).
- **E3 (step 7) — a mandatory attribute is missing.** Course, topic, difficulty, positive marks and negative marks are all mandatory (FR-008).
- **E4 (step 8) — a delete is attempted on a question inside an attempted examination.** Refused at the database, not only in the UI, so a direct write cannot bypass it. IT_014

Business rules

- **BR-01.** A question in a published or attempted examination is immutable; a change produces a new version (FR-009). This is what makes a result defensible after the fact.
- **BR-02.** Difficulty is one of exactly three values — Easy, Medium, Hard.
- **BR-03.** Positive and negative marks live on the *question*; whether negative marking is applied at all lives on the *examination* (FR-010). Both must agree before a score is meaningful.
- **BR-04.** A question belongs to exactly one course, which is what scopes the bank.

Test cases: UT_022–UT_027, IT_012–IT_014, ST_005

UC-06 — Assemble and publish an examination

Use case ID	UC-06
Primary actor	Faculty
Requirements	FR-010, FR-011, FR-012
Includes	UC-05 (question selection), UC-13 (audit entry)
Stakeholders	*Faculty* — wants a paper assembled from existing questions with the policy set once. *Student* — must never see a Draft.
Trigger	Faculty selects **New Examination** within a course.
Pre-conditions	Faculty is signed in and assigned to the course. The course's question bank holds the questions to be used.
Post-conditions (success)	The examination exists in **Published** state with its questions, its duration, its declared total marks, its instructions and its policy values. An audit entry is written.
Post-conditions (failure)	The examination remains in **Draft**, or is not created at all. It is invisible to Students either way.
Frequency	A few times per course per semester.

Main success scenario

39. Faculty selects **New Examination** within a course.
40. The system presents the examination form.
41. Faculty enters the title, the duration in minutes, the declared total marks and the free-text instructions.
42. Faculty sets the policy values: whether negative marking is enabled and its value, whether questions and options are shuffled, and the pass mark.
43. The system creates the examination in **Draft** state.
44. Faculty opens the question selector and picks questions from the course's bank (UC-05).
45. The system adds each selected question to the paper, holding a reference to the exact version selected.
46. Faculty reviews the assembled paper.
47. Faculty selects **Publish**.
48. The system moves the examination to **Published** state and writes an audit entry.
49. The examination becomes eligible for scheduling (UC-07). It is not yet visible to any Student, because visibility also requires a schedule.

Alternate flows

- **A1 (step 6) — remove a question from the paper.** Permitted while the examination is in Draft. The question itself is untouched in the bank.
- **A2 (step 8) — save and return later.** The examination stays in Draft indefinitely and remains invisible to Students throughout (FR-012).
- **A3 (step 4) — negative marking disabled.** An incorrect response then scores exactly zero rather than a negative value. UT_056

Exception flows

- **E1 (step 9) — publish is attempted with no questions attached.** Refused; an empty paper cannot be published.
- **E2 (step 6) — a question from a different course is selected.** Refused; the selector is scoped to the examination's own course.
- **E3 (step 9) — a Student or a Faculty member not assigned to the course calls the publish endpoint.** HTTP 403 (FR-003).
- **E4 (any step) — a Student requests a Draft examination by identifier.** HTTP 403 and no metadata is returned; Draft is a state, not a hidden link. IT_016

Business rules

- **BR-01.** Only **Published** examinations are visible to Students; a Draft is invisible whatever its schedule (FR-012).
- **BR-02.** The paper references a specific *version* of each question, which is what makes FR-009 effective.
- **BR-03.** Policy values — negative marking, shuffling and the pass mark — are per examination, set at authoring and configurable at run time, never hard-coded (NFR-010).
- **BR-04.** The declared total marks are recorded as entered. A check that they equal the sum of the question marks is out of scope for this version; ensuring they match is Faculty's responsibility.

Test cases: UT_028–UT_033, IT_015–IT_016, ST_006

UC-07 — Schedule an examination and surface it on the Student dashboard

Use case ID	UC-07
Primary actor	Faculty (scheduling); Student (viewing)
Requirements	FR-013, FR-014
Stakeholders	*Faculty* — wants a fixed window the whole cohort sits within. *Student* — wants to know what is coming, what is live now, and what is finished.
Trigger	Faculty opens a published examination and selects **Schedule**; or a Student opens their dashboard.
Pre-conditions	The examination is **Published** (UC-06). Students are enrolled in the course (UC-04).
Post-conditions (success)	The examination carries a window start and a window end, and appears on every enrolled Student's dashboard under Upcoming, Live or Completed according to the server clock.
Post-conditions (failure)	The examination has no window and appears on no Student dashboard.
Frequency	Once per examination, with occasional rescheduling before the window opens.

Main success scenario

50. Faculty opens a published examination and selects **Schedule**.
51. Faculty sets the window start date-time and the window end date-time.
52. The system validates that the end is after the start, and persists the window.
53. The system writes an audit entry.

54. Each enrolled Student opens their dashboard.
55. The system lists that Student's examinations under three headings, derived from the **server** clock against each window: **Upcoming** before the start, **Live** between start and end, **Completed** after the end or after the Student has submitted.
56. A Live examination presents a **Start** control (UC-08).

Alternate flows

- **A1 (step 2) — reschedule before the window opens.** Faculty changes either boundary; the dashboards of all enrolled Students reflect the change on their next load, and an audit entry records both the previous and the new values.
- **A2 (step 6) — the Student has already submitted while the window is still open.** The examination moves to **Completed** for that Student alone; the cohort still sees it as Live.
- **A3 (step 6) — the Student is enrolled in no course.** The dashboard renders with an empty state, not an error.

Exception flows

- **E1 (step 3) — the window end is at or before the window start.** Rejected; no window is persisted.
- **E2 (step 1) — scheduling is attempted on a Draft examination.** Refused; a schedule presupposes publication (FR-012).
- **E3 (step 6) — a Student requests the dashboard feed for a course they are not enrolled in.** HTTP 403 (FR-006).

Business rules

- **BR-01.** Upcoming, Live and Completed are computed from the server clock. A client clock has no effect on any of the three (FR-017).
- **BR-02.** A Student's dashboard shows only the courses that Student is enrolled in (FR-006, FR-014).
- **BR-03.** The window is not validated against the examination duration in this version. A window shorter than the duration is therefore possible and is Faculty's responsibility.

Test cases: UT_034–UT_038, IT_017–IT_018, ST_007

UC-08 — Start an attempt and receive the paper

Use case ID	UC-08
Primary actor	Student
Secondary actors	Clock
Requirements	FR-015, FR-016, FR-017, FR-018
Includes	UC-11 (on submission)
Extended by	UC-09 *(session interrupted)*, UC-10 *(remaining time reaches zero)*, UC-13 *(window loses focus)*
Stakeholders	*Student* — wants the full duration, reliable navigation and no lost work. *Faculty* — wants every candidate to get the same duration and no

	candidate to see the key.
Trigger	The Student selects **Start** on a Live examination.
Pre-conditions	The Student is signed in and enrolled. The examination is Published and its window is currently open by the server clock. The Student has no previously submitted attempt for it.
Post-conditions (success)	An attempt record exists, stamped with the server start time. The paper is delivered without any answer-key data. The countdown runs from server time.
Post-conditions (failure)	No attempt record is created, and no question payload is delivered.
Frequency	Once per Student per examination.

Main success scenario

57. The Student opens a Live examination and selects **Start**.
58. The system checks, against the server clock, that the current time lies inside the window, that the Student is enrolled, and that no submitted attempt already exists for this Student and examination.
59. The system creates the attempt record, stamped with the **server** start time.
60. Only after that record exists does the system deliver the question payload. `IT_019`
61. The system strips every correct-answer field from the payload — no key, no `isCorrect`, nothing in the client-side state (`FR-029`). `IT_035`
62. The system displays exactly one question at a time, together with the navigation palette.
63. The palette shows every question in one of four states: **Not Visited**, **Not Answered**, **Answered**, **Marked for Review**.
64. The system displays a countdown whose remaining time is derived from the server clock and the stored start time, never from the client clock.
65. The Student navigates and answers; each response is persisted automatically (UC-09).
66. The Student selects **Submit** and confirms.
67. The system records the submission with a server timestamp, moves the attempt to **Submitted**, and refuses every further response.
68. The system evaluates the attempt automatically (UC-11).

Alternate flows

- **A1 (step 9) — the Student marks a question for review.** The palette state becomes *Marked for Review*; the response, if any, is retained. `UT_043`
- **A2 (step 9) — the Student clears a response.** The stored response is emptied and the palette returns the question to *Not Answered*. `UT_047`
- **A3 (step 9) — the session is interrupted.** UC-09 applies at the extension point *session interrupted*.
- **A4 (step 10) — the Student never submits and the timer expires.** UC-10 applies at the extension point *remaining time reaches zero*.
- **A5 (step 9) — the examination window loses focus.** UC-13 applies at the extension point *window loses focus*. The attempt itself is not interrupted; the event is recorded.

Exception flows

- **E1 (step 2) — the window has not opened.** Refused with *"This examination has not opened yet."* No attempt record is created. UT_039
- **E2 (step 2) — the window has closed.** Refused with *"This examination has closed."* UT_040
- **E3 (step 2) — a submitted attempt already exists.** Refused with *"You have already submitted this examination."* No second attempt is created. UT_041
- **E4 (step 2) — the Student is not enrolled.** HTTP 403 (FR-006).
- **E5 (step 8) — the client clock is altered.** The remaining time is unaffected, because it is derived server-side. UT_042, IT_020

Business rules

- **BR-01.** The attempt record is created **before** the paper is delivered, so a paper can never be seen without a timed attempt existing to account for it (FR-016). IT_019
- **BR-02.** Every timing decision uses the server clock (FR-017), which is why NFR-004 requires NTP synchronisation.
- **BR-03.** Exactly one question stem is present in the delivered markup; the remaining stems are not merely hidden with CSS. UT_044
- **BR-04.** Palette state is derived from server-held response data, so it survives a reload unchanged. IT_021
- **BR-05.** One attempt per Student per examination.

Test cases: UT_039–UT_044, IT_019–IT_021, ST_008

UC-09 — Autosave and resume after interruption

Use case ID	UC-09
Primary actor	Student
Requirements	FR-019, FR-021
Extends	UC-08, at extension point *session interrupted*
Stakeholders	*Student* — must not lose answers to a dropped connection or a crashed browser. *Faculty* — must not have a resumption hand back extra time.
Trigger	Autosave: the Student selects, changes or clears a response. Resume: the Student reopens an attempt that is still in progress.
Pre-conditions	An attempt is in progress and has not been submitted.
Post-conditions (success)	Every response is on the server within 5 seconds of selection. On resume, all persisted responses are restored and the remaining time is computed from the original start time.
Post-conditions (failure)	The Student is told the response is unsaved and the client retries; no response is silently dropped.
Frequency	Autosave: continuously. Resume: occasionally.

Main success scenario

69. The Student selects an option on a question.
70. Without any explicit save action — there is no Save control — the client sends the response to the server.
71. The server commits it within 5 seconds of the selection and acknowledges. IT_022
72. The palette updates that question to *Answered*.

73. *(Resume)* The session is interrupted — the network drops, the browser is closed or the device changes.
74. The Student signs in again and reopens the examination.
75. The system recognises the in-progress attempt and restores every persisted response.
76. The system computes the remaining time from the **original** server start time and the configured duration, and resumes the countdown from there.
77. The Student continues from where they were.

Alternate flows

- **A1 (step 1) — the Student changes an existing response.** The new value replaces the old; exactly one response is held per question. UT_046
- **A2 (step 1) — the Student clears a response.** The cleared state is persisted and the palette returns to *Not Answered*. UT_047
- **A3 (step 6) — the Student resumes on a different device or browser profile.** The same attempt is restored, with the same responses and the same remaining time, because the state is server-held and not in browser storage. IT_025
- **A4 (step 6) — the duration has already elapsed during the interruption.** UC-10 has already auto-submitted the attempt; the Student is shown the submitted state, not a resumable one. IT_026

Exception flows

- **E1 (step 3) — the persistence request fails or times out.** The client retries. The response is not treated as saved until the server acknowledges, and the Student is shown an unsaved indication.
- **E2 (step 5) — the browser process is killed without warning.** Every response already acknowledged is intact on resume; nothing depended on an unload handler. IT_023

Business rules

- **BR-01.** No Save control exists. Persistence is automatic within 5 seconds (FR-019). UT_045
- **BR-02.** Resumption never grants extra time. Remaining time is always `start + duration − now` on the server clock (FR-021). UT_048, IT_024
- **BR-03.** Responses live on the server, never only in `localStorage`, which is what makes A3 work and what makes NFR-004 recoverable.
- **BR-04.** After an unplanned server restart, in-progress attempts remain resumable with responses intact and the correct remaining time (NFR-004).

Test cases: UT_045–UT_048, IT_022–IT_025, ST_009

UC-10 — Automatic submission on timer expiry

Use case ID	UC-10
Primary actor	Clock
Requirements	FR-020

Extends	UC-08, at extension point *remaining time reaches zero*
Includes	UC-11
Stakeholders	*Student* — wants everything answered so far to count. *Faculty* — wants the duration enforced identically for everyone, whatever the candidate's browser is doing.
Trigger	The remaining time of an in-progress attempt reaches zero on the server clock.
Pre-conditions	An attempt is in progress and has not been submitted.
Post-conditions (success)	The attempt is **Submitted**, with a server timestamp equal to the start plus the duration. Every response persisted up to that instant is retained. No further response is accepted. The attempt enters evaluation.
Post-conditions (failure)	None — expiry is enforced server-side and cannot be declined.
Frequency	Whenever a candidate does not submit in time.

Main success scenario

78. The server clock reaches the attempt's start time plus its configured duration.
79. The system submits the attempt automatically, with no action by the Student.
80. The system retains every response persisted up to that instant; unanswered questions are recorded as unattempted.
81. The system sets the attempt state to **Submitted** and stamps a server-generated submission time.
82. The system makes the attempt screen read-only and displays *"Time is up. Your paper has been submitted."*
83. The system refuses every further response for that attempt.
84. The system evaluates the attempt automatically on the same path as a manual submission (UC-11). `IT_028`

Alternate flows

- **A1 (step 1) — the Student submits manually first.** This use case does not run; UC-08 step 10 applies instead.
- **A2 (step 5) — the browser was closed before expiry.** The attempt is still submitted at the true expiry. The Student sees the submitted state when they next sign in. `IT_026`

Exception flows

- **E1 (step 6) — a response-save call arrives for the attempt after submission.** Refused with *"This attempt has already been submitted."* `UT_051`
- **E2 (step 1) — the client clock is set back to defer expiry.** Ineffective. Submission occurs at the true server expiry and later answers are refused. `IT_027`

Business rules

- **BR-01.** Expiry is enforced on the server. The client countdown is a display, never the control (`FR-017`, `FR-020`).
- **BR-02.** Auto-submission retains, never discards, what was already persisted. `UT_050`

- **BR-03.** The submission timestamp equals the attempt start plus the configured duration. UT_052
- **BR-04.** An auto-submitted attempt is indistinguishable from a manual one downstream — same evaluation path, same result shape. This is what stops a candidate gaining or losing anything by letting the clock run out.

Test cases: UT_049–UT_052, IT_026–IT_028, ST_010

UC-11 — Evaluate a submitted attempt automatically

Use case ID	UC-11
Primary actor	None — this use case has no actor and no human trigger
Requirements	FR-022, FR-023, FR-024
Included by	UC-08, UC-10
Stakeholders	*Student* — wants a score that is correct and arrives without a wait. *Faculty* — wants no manual grading at all.
Trigger	An attempt reaches the **Submitted** state, whether by the Student or by expiry.
Pre-conditions	The attempt is Submitted. The examination's answer key and policy values are available.
Post-conditions (success)	The attempt carries a total score, a percentage and a Pass or Fail outcome, plus per-question marks. The result is stored but **withheld** from the Student until UC-12 publishes it.
Post-conditions (failure)	The attempt is left flagged for evaluation and retried; it is never silently left unscored.
Frequency	Once per submitted attempt.

Main success scenario

85. An attempt enters the Submitted state.
86. The system loads the examination's answer key and its policy values — negative marking on or off and its value, and the pass mark.
87. For each question on the paper the system compares the stored response against the key.
88. A correct response is awarded the question's full positive marks. UT_053
89. An incorrect response is deducted the configured negative marks. UT_054
90. An unattempted question is awarded zero, with **no** deduction. UT_055
91. The system sums the per-question marks into the total score.
92. The system computes the percentage against the examination's declared total marks.
93. The system compares the percentage against the pass mark and records **Pass** or **Fail**.
94. The system stores the evaluation with its per-question breakdown.
95. The result remains invisible to the Student until Faculty publishes it (UC-12, FR-025).

Alternate flows

- **A1 (step 5) — negative marking is disabled on the examination.** An incorrect response is awarded exactly zero, not a negative value. UT_056
- **A2 (step 9) — the percentage equals the pass mark exactly.** The outcome is **Pass**; the boundary is inclusive. UT_057

Exception flows

- **E1 (step 2) — the answer key cannot be resolved for a question.** Evaluation of the attempt is aborted rather than completed with a guessed key; the attempt is flagged for retry and no partial score is stored.
- **E2 (step 10) — evaluation exceeds the 5-second budget for a 100-question attempt.** A performance defect against NFR-002, not a functional failure; the result is still stored. IT_030

Business rules

- **BR-01.** Evaluation is automatic and unconditional. No Faculty trigger, no approval step, no attempt left waiting (FR-022). IT_029
- **BR-02.** Skipping and answering wrongly are different outcomes. Zero for unattempted is the boundary that makes a negative-marking paper fair, and it is tested explicitly. UT_055
- **BR-03.** The per-question marks always sum exactly to the reported total. UT_063
- **BR-04.** Evaluation is deterministic — the same responses against the same key and policy always produce the same score. ST_011 hand-computes two attempts to confirm this.
- **BR-05.** Two conventions remain open with the instructor and are recorded in §8 of the Test Plan: whether the negative deduction is flat or a fraction of the question's marks, and whether the pass boundary is inclusive. The flows above assume **flat** and **inclusive**.

Test cases: UT_053–UT_058, IT_029–IT_031, ST_011

UC-12 — Publish results and display the scorecard

Use case ID	UC-12
Primary actor	Faculty (publishing); Student (viewing)
Requirements	FR-025, FR-026, FR-027
Includes	UC-13 (audit entry)
Stakeholders	*Faculty* — wants to release results only when the cohort has finished and the key is confirmed. *Student* — wants a per-question breakdown, not just a number.
Trigger	Faculty selects **Publish Results** on an examination; or a Student opens an examination whose results are published.
Pre-conditions	The attempts are submitted and evaluated (UC-11).
Post-conditions (success)	Every enrolled Student who attempted can see their own scorecard: total, percentage, Pass or Fail, and per-question marks.
Post-conditions (failure)	Results remain withheld. Stored evaluations are unchanged either way.
Frequency	Once per examination, occasionally reversed.

Main success scenario

96. Faculty opens an examination whose attempts have been evaluated.
97. The system shows the cohort's evaluation state.
98. Faculty selects **Publish Results**.
99. The system marks the examination's results as published and writes an audit entry.
100. A Student opens the examination from their dashboard.
101. The system displays the scorecard: the total score, the percentage, the Pass or Fail outcome, and the marks obtained on each question.
102. The per-question marks displayed sum exactly to the displayed total. UT_063

Alternate flows

- **A1 (step 5) — results are not yet published.** The Student sees *"Results not yet published"* and no score, percentage or outcome anywhere on the page. UT_059
- **A2 (step 3) — Faculty unpublishes.** The scorecard disappears for every Student and the examination reverts to *"Results not yet published."* The stored evaluation is unchanged, so republishing restores exactly the same figures. UT_061

Exception flows

- **E1 (step 5) — a Student calls the result endpoint directly while results are unpublished.** HTTP

103. No score, percentage or per-question mark appears anywhere in the response body — the withholding is an access control, not a UI condition. IT_032

- **E2 (step 5) — a Student calls the result endpoint with another Student's attempt identifier.** HTTP 403; the other Student's score is not disclosed. IT_033
- **E3 (step 3) — a Student calls the publish endpoint.** HTTP 403 (FR-003).

Business rules

- **BR-01.** An evaluated result is withheld until Faculty publishes it (FR-025). Evaluation and disclosure are deliberately separate events.
- **BR-02.** Publication is reversible (FR-026), and reversing it never alters a stored score.
- **BR-03.** A Student sees their own result and no one else's. Cohort statistics are not in the reduced baseline.
- **BR-04.** Every displayed figure equals the stored evaluation exactly; the scorecard recomputes nothing at display time. IT_034

Test cases: UT_059–UT_063, IT_032–IT_034, ST_012

UC-13 — Log integrity events and maintain the audit trail

Use case ID	UC-13

Primary actor	Administrator (reviewing)
Requirements	FR-028, FR-029, FR-030
Extends	UC-08, at extension point *examination window loses focus*
Included by	UC-03, UC-05, UC-06, UC-12
Stakeholders	*Administrator* — needs a trail that can be reconciled after the fact. *Faculty* — wants evidence, not an automated verdict. *Student* — is entitled to be judged by a person, not by a focus event.
Trigger	Integrity: an active attempt's window loses focus. Audit: any creation, modification or deletion of a user, question, examination, answer key or result.
Pre-conditions	For integrity: an attempt is in progress. For audit: a mutating operation is committing.
Post-conditions (success)	The event or the mutation is recorded with a server-side timestamp in an append-only log, reconcilable against the examination cycle.
Post-conditions (failure)	The mutation itself does not commit — an unlogged mutation is not an acceptable outcome.
Frequency	Integrity: sporadic. Audit: on every mutation.

Main success scenario — integrity events

104. A Student's attempt is in progress.
105. The examination window loses focus — a tab switch, an application switch, a full-screen exit.
106. The system records one focus-loss event against that attempt, carrying a **server-side** timestamp.
107. The attempt continues uninterrupted. Nothing is auto-submitted and no verdict is reached.
108. Faculty and the Administrator can read the events against the attempt after the fact.

Main success scenario — audit trail

109. An actor creates, modifies or deletes a user, a question, an examination, an answer key or a result.
110. Within the same transaction, the system appends an audit entry recording the actor, the action, the affected entity, a server timestamp, and the previous and new values. UT_068
111. The entry is immutable thereafter; the application exposes no path to update or delete it.
112. The Administrator opens the audit log, filters it to an entity or an examination, and reconciles the trail against what happened.

Alternate flows

- **A1 — focus is lost repeatedly.** Each loss is recorded as its own event with its own distinct timestamp; none is coalesced or dropped. UT_065
- **A2 — focus is never lost.** The event log for that attempt is empty. The detector does not fire on ordinary in-page interaction. UT_066

Exception flows

- **E1 — the client supplies a tampered timestamp.** Ignored. The stored timestamp is the server's. UT_067

- **E2 — an update or delete is attempted against an audit row.** Refused, both through the application and by the table's own constraints. `IT_037`
- **E3 — the audit write fails.** The enclosing mutation is rolled back. A mutation that cannot be logged does not happen.

Business rules

- **BR-01.** The system is **evidentiary, not judicial**. It logs and flags; a human decides. There is no automatic penalty, no auto-submission on a violation count, and no Proctor actor.
- **BR-02.** No correct-answer data reaches the candidate during an active attempt — not in a payload, not in the rendered HTML, not in client-side JavaScript state (FR-029). `IT_035`
- **BR-03.** Every one of the five mutation classes writes an entry; none is silently unlogged. `IT_036`
- **BR-04.** The audit log is append-only. `IT_037`
- **BR-05.** All integrity and audit timestamps are server-side, which is why NFR-004 requires an NTP-synchronised clock.

Test cases: `UT_064`–`UT_068`, `IT_035`–`IT_037`, `ST_013`

6. Assumptions, Dependencies and Open Items

Assumptions and dependencies. Students have a personal device with a supported browser and an internet connection · examinations are attempted remotely · the server clock is synchronised by NTP, since every timing rule depends on it · the system is a single Django application with an SQLite database, deployed on one Linux virtual machine for the demonstration (see `Documents/SAD.md`).

Open items awaiting the course instructor; the assumption taken is used throughout this SRS and the Test Plan:

Open question	Affects	Assumption taken
Negative marking — a flat deduction per wrong answer, or a fraction of the question's marks	FR-023, UC-11	**Flat** deduction of the question's configured negative marks
Is a score exactly at the pass mark a Pass?	FR-024, UC-11	**Inclusive** — 40.0% against a 40% pass mark is a Pass

7. Traceability

Artefact	Where
Backlog — one GitHub issue per FR / NFR, with story points, assignee and sprint	GitHub Project *Online Exam Portal - Backlog*
Sprint 1 and Sprint 2 plan	`Documents/Sprint_Plan.md`
Architecture — components realising each requirement group	`Documents/SAD.md`
Test cases for every functional requirement (118 cases, 13 use cases)	`Documents/Test_Plan.md`